

1.10. Un Mot fait de variables créées d'avance

`x`, `y` et le `texte` sont créés d'abord ; le `Mot` les rassemble sous un seul nom.

1. Ce qu'il faut comprendre

On crée d'abord `x`, `y` et `MOT`, **avant** le `Mot` : il ne peut se servir que de ce qui existe déjà.

`Mot mot_xy = { x, y, MOT };` ne recopie pas 5 et 6 : il **relit** `x` et `y` à chaque `texte()`. Changer `x` déplace donc le mot.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Mot>` → [1.7. Mot — un texte et sa place, sous un nom](#)
- `#include <texte>` → [0.0.2. La fonction `texte\(\)` — écrire un mot](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **350 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      57 o ← la console, à l'allumage – la mise en route de la console
main()         25 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
AdresseCase    10 o ← texte() – trouve l'adresse d'une case de l'écran
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
CopierTuiles   67 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
Tuiles         96 o ← CopierTuiles – 6 lettres sur 43
"BONJOUR"      7 o ← texte() – un texte gravé
```

Lettres gravées : BJNORU

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
texte() × 1    18 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <Mot>      // un texte et sa place, sous un seul nom
#include <texte>    // écrit un texte à l'écran
```

2. UNE VARIABLE — X

```
uint8_t x = 5;
```

3. UNE VARIABLE — Y

```
uint8_t y = 6;
```

4. UN TEXTE NOMMÉ — MOT

```
const char MOT[] = "BONJOUR";
```

5. UNE VARIABLE — MOT_XY

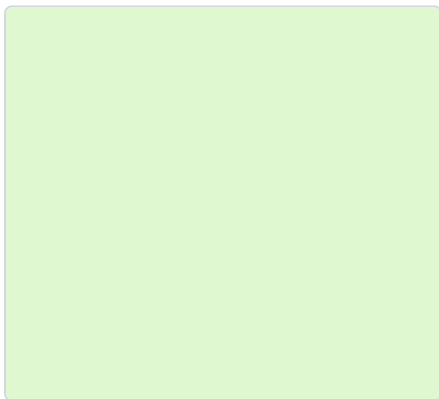
```
Mot mot_xy = { x, y, MOT };
```

6. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {  
    texte(mot_xy);  
  
    while (true) image();  
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de commencer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

« BONJOUR » en (x, y) : colonne 5, ligne 6.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ x vaut 5 et y vaut 6

✓ « BONJOUR » est écrit en (5, 6)

